Web Engineering & Development (SWE 363)

Back-end Development Fundamentals

In today's lecture:

- Express.js Request data
  - Query String Parameters
  - Route Parameters
  - Param Middleware
  - Request Body
  - Sending JSON Responses
  - Status Codes

Reference:

- Zybook: 6.5

6.5 Express request data

How do we get data from the client?

When a client makes a request to our server, they often send **data**:

- **Query parameters**: `/search?q=react&page=2`
- **Route parameters**: `/users/123` or `/profile/John/Doe`
- **Request body**: JSON data, form data, etc.

Today we'll learn how to **read** this data in Express!

Three ways to send data

Method	Location	Example	Use Case
Query String	URL	`/search?q=react&page=2`	Optional filters, search terms
Route Parameters	URL path	`/users/123`	Identifying specific resources
Request Body	HTTP body	POST with JSON	Creating/updating data

1. Query String Parameters

What are Query String Parameters?

Query strings are key-value pairs added to URLs after `?`

```
http://localhost:3000/search?q=react&page=2&sort=date
```

Format: `?key1=value1&key2=value2&key3=value3`

Why use Query Strings?

Optional - can be added or removed easily

Bookmarkable - URLs can be saved/shared

Cacheable - browsers can cache different query combinations

Flexible - don't change the resource path

Common uses: Search terms, filters, pagination, sorting

Reading Query Parameters in Express

Use `req.query` to access query string parameters

```
app.get("/search", (req, res) => {
  const { q, page, sort } = req.query;
  // q = "react"
  // page = "2"
  // sort = "date"

  res.json({ query: q, page, sort });
});
```

Important: Values are always **strings**!

Query Parameters: Important Notes

Values are strings - convert if needed:

```
const page = Number(req.query.page) || 1;
```

Always validate - check if required params exist:

```
if (!req.query.name) {
  return res.status(400).json({ error: "name required" });
}
```

Optional by nature - missing params are `undefined`

2. Route Parameters

What are Route Parameters?

Route parameters are **named placeholders** in the URL path

```
app.get("/users/:userId", (req, res) => {
  // :userId is a route parameter
});
```

URL: `/users/123`

Parameter: `userId = "123"`

They identify **specific resources** in the path!

Why use Route Parameters?

Clean URLs - `/users/123` vs `/users?id=123`

RESTful - follows REST API conventions

Descriptive - makes the resource clear

Hierarchical - supports nested resources

Example: `/orders/987/items/2` (order 987, item 2)

Reading Route Parameters

Use `req.params` to access route parameters

```
app.get("/users/:userId", (req, res) => {
  const { userId } = req.params;
  // userId = "123" (string)

  res.json({ userId });
});
```

URL: `/users/123` → `userId = "123"`

Multiple Route Parameters

You can have multiple parameters in one route:

```
app.get("/profile/:first/:last", (req, res) => {
  const { first, last } = req.params;

  res.json({
    ok: true,
    fullName: `${first} ${last}`
  });
});
```

URL: `/profile/Jack/Black`

Result: `{ ok: true, fullName: "Jack Black" }`

Route Parameters: Important Notes

Values are strings - convert to numbers if needed:

```
const userId = Number(req.params.userId);
```

Required by route - if missing, route won't match

Order matters - `/users/:id/posts/:postId` vs `/users/:postId/posts/:id`

Query vs Route Parameters

Feature	Query Parameters	Route Parameters
Syntax	`?key=value`	`/:paramName`
Required?	Optional	Required (for route match)
Use Case	Filters, search, pagination	Resource identification
Example	`/search?q=react`	`/users/123`
Access	`req.query`	`req.params`

3. Route Parameter Middleware

The Problem: Repeated Validation

What if multiple routes use the same parameter? **Problem**: Code duplication!

```
app.get("/users/:userId", (req, res) => {
  const userId = Number(req.params.userId);
  if (!Number.isFinite(userId) || userId < 1) {
    return res.status(400).json({ error: "Invalid userId" });
  }
  // ... use userId
});

app.delete("/users/:userId", (req, res) => {
  const userId = Number(req.params.userId);
  if (!Number.isFinite(userId) || userId < 1) {
    return res.status(400).json({ error: "Invalid userId" });
  }
  // ... same validation again!
});
```

Solution: Param Middleware

`app.param()` runs **once** when a route with that parameter matches

```
app.param("userId", (req, res, next, userId) => {
  const num = Number(userId);
  if (!Number.isFinite(num) || num < 1) {
    return res.status(400).json({
      ok: false,
      error: "userId must be positive number"
    });
  }
  req.userIdNum = num; // Store validated value
  next(); // Continue to route handler
});
```

Runs automatically for any route with `:userId` !

Using Param Middleware

After defining param middleware, routes can use the validated value:

```
// Middleware (runs first)
app.param("userId", (req, res, next, userId) => {
  const num = Number(userId);
  if (!Number.isFinite(num) || num < 1) {
    return res.status(400).json({
      ok: false, error: "userId must be positive number"
    });
  }
  req.userIdNum = num;
  next();
});
// Route (uses validated value)
app.get("/users/:userId", (req, res) => {
  res.json({ ok: true, userId: req.userIdNum });
});
```

Param Middleware: How It Works

1. **Request comes in**: `/users/42`
2. **Express matches route**: `GET /users/:userId`
3. **Param middleware runs**: `app.param("userId", ...)`
   - Validates and converts `"42"` → `42`
   - Stores in `req.userIdNum`
   - Calls `next()`
4. **Route handler runs**: Uses `req.userIdNum`

One validation, multiple routes benefit!

Param Middleware: Benefits

DRY - Don't Repeat Yourself

Centralized validation - one place to fix bugs

Consistent - all routes get same validation

Reusable - works for GET, POST, DELETE, etc.

Best for: Type conversion, validation, normalization

4. Receiving JSON Data

Request Body: Another Way to Send Data

So far we've seen data in URLs (query strings, route params)

Request body is data sent in the HTTP request body:

- **JSON**: `{ "name": "Ali", "age": 22 }`
- **Form data**: `name=Ali&age=22`
- **Files**: multipart/form-data

Common for **POST** and **PUT** requests

Why Use Request Body?

URLs have limits:

- Query strings get long and messy
- Route params are for resource IDs
- Sensitive data shouldn't be in URLs

Request body is better for:

- Large amounts of data
- Complex objects
- Sensitive information (passwords, etc.)
- Creating/updating resources

Receiving JSON in Express

To receive JSON data, you need **middleware** to parse it:

```
import express from "express";
const app = express();

// Enable JSON parsing middleware
app.use(express.json());

// Now you can read JSON from req.body
app.post("/users", (req, res) => {
  const { name, age } = req.body;
  // name = "Ali"
  // age = 22
  res.json({ ok: true, user: { name, age } });
});
```

Important: `app.use(express.json())` must come **before** your routes!

Receiving URL-Encoded Data

For traditional HTML forms, use `express.urlencoded()`:

```
app.use(express.urlencoded({ extended: true }));

app.post("/login", (req, res) => {
  const { username, password } = req.body;
  // Data comes from form: username=john&password=secret
  // ...
});
```

Difference:

- `express.json()` - parses JSON bodies
- `express.urlencoded()` - parses form data (`key=value&key2=value2`)

Request Body: Important Notes

Always validate - check required fields:

```
if (!req.body.name) {
  return res.status(400).json({ error: "name required" });
}
```

Middleware order matters - `app.use(express.json())` before routes

Data types - JSON preserves types (numbers, booleans, etc.)

Security - validate and sanitize all input!

Summary: All Ways to Get Data

Source	Access	Example	Use Case
Query String	`req.query`	`/search?q=react`	Optional filters
Route Params	`req.params`	`/users/123`	Resource ID
Request Body	`req.body`	POST with JSON	Creating/updating data

5. Sending JSON Responses

Sending JSON in Express

Use `res.json()` to send JSON responses:

```
app.get("/data", (req, res) => {
  res.json({ ok: true, message: "Success!" });
});
```

Automatically:

- Sets `Content-Type: application/json`
- Converts object to JSON string
- Sends response

Setting Status Codes

Use `res.status()` before `res.json()`:

```
// Success (200 is default)
res.json({ ok: true, data: result });

// Error (400 Bad Request)
res.status(400).json({
  ok: false,
  error: "Invalid input"
});

// Not Found (404)
res.status(404).json({
  ok: false,
  error: "Resource not found"
});
```

Common Status Codes

Code	Meaning	Use Case
`200`	OK	Successful request
`400`	Bad Request	Invalid input/parameters
`404`	Not Found	Resource doesn't exist
`500`	Server Error	Server-side error

Always include status codes for errors!

Summary: Request Data in Express

Type	Access	Example	Use Case
Query	`req.query`	`/search?q=react`	Optional filters
Route	`req.params`	`/users/123`	Resource ID
Body	`req.body`	POST with JSON	Creating/updating data
Param Middleware	`app.param()`	Validates route params	Reusable validation

30m

Demo

6.4 Request Data in Express